

Lab 11 – Intermediate Code Generation (Quadruple, Triple, Indirect Triple)

Aim

To generate quadruple, triple, and indirect triple representations for a given arithmetic expression.

Algorithm

1. Start the program.
2. Read an arithmetic expression.
3. Scan the expression from left to right.
4. Identify operators according to precedence (*, /, +, -).
5. For each operation:
 1. Generate a temporary variable.
 2. Store operator, operand1, operand2, and result.
6. Replace the evaluated subexpression with the temporary variable.
7. Continue until the complete expression is reduced.
8. Display:

1. **Quadruple**
2. **Triple**
3. **Indirect Triple**

9. Stop.

C Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char expr[50];
    char op[10], arg1[10], arg2[10], res[10];
    int count = 0;
    int i;

    printf("Enter expression (Example: a=b+c*d): ");
    scanf("%s", expr);

    for (i = 0; expr[i] != '\0'; i++) {
        if (expr[i] == '*' || expr[i] == '/' ||
            expr[i] == '+' || expr[i] == '-') {

            op[count] = expr[i];
            arg1[count] = expr[i - 1];
            arg2[count] = expr[i + 1];
            res[count] = 'A' + count;
```

```

        expr[i + 1] = res[count];
        count++;
    }
}

printf("\nQuadruple\n");
printf("No\tOp\tArg1\tArg2\tResult\n");

for (i = 0; i < count; i++) {
    printf("%d\t%c\t%c\t%c\t%c\n",
        i,
        op[i],
        arg1[i],
        arg2[i],
        res[i]);
}

printf("\nTriple\n");
printf("No\tOp\tArg1\tArg2\n");

for (i = 0; i < count; i++) {
    printf("%d\t%c\t%c\t%c\n",
        i,
        op[i],
        arg1[i],
        arg2[i]);
}

printf("\nIndirect Triple\n");
printf("Pointer\tStatement\n");

for (i = 0; i < count; i++) {
    printf("%d\t(%d)\n", i, i);
}

return 0;
}

```

Sample Input

a=b+c*d

Sample Output

Quadruple

No	Op	Arg1	Arg2	Result
0	*	c	d	A
1	+	b	A	B

Triple

No	Op	Arg1	Arg2
0	*	c	d
1	+	b	A

Indirect Triple

Pointer Statement

0	(0)
1	(1)

Result

Thus the quadruple, triple, and indirect triple representations were generated successfully for the given expression.